The Structure of Information Networks	**Homework 1**
CS/INFO 6850 Fall 2024	**Due 6pm, Wednesday, Sept. 18, 2024**

The goal of this problem set is to provide practice implementing some basic network analysis techniques on a moderate-sized network dataset — specifically, a coauthorship network constructed from a bibliography file of computer science papers maintained by Joel Seiferas at the University of Rochester. To make sure that the bibliography file is easy to obtain, a copy of it is has been posted on Canvas as the file "ps1data.txt". As a first step, we will only be using papers written between the years 1985 and 2005 inclusive, so you should only keep lines in the file corresponding to papers written in this interval.

Building the coauthorship network. Here are some instructions on how to create the coauthorship network from the raw bibliography. The short explanation is: each line represents a paper, and we want to build the undirected graph whose nodes are the people named in the bibliography, and whose edges join those pairs of people who've coauthored a paper in the bibliography. This graph should only be constructed using papers written between the years 1985 and 2005 inclusive.

The more detailed instructions now follow. Each line in the bibliography describes a distinct paper, and has the following format:

```
year [number] conference/journal author & author & ...  & author, title
```

Here, `conference/journal` is an acronym encoding the conference or journal where the paper appeared, `year` is the year of the paper, and `number` is the volume number of the journal or conference. We write `number` in brackets above because it is present in some lines and (when it is not known or not applicable) absent in others. Authors are given by last name only, and separated by the & symbol. The list of authors ends with a comma, and the the remainder of the line is the title. Thus, a sample line from the file is

```
1994 11 ALGRTHMICA Khuller & Naor, Flow in Planar Graphs with Vertex Capacities
```

encoding the paper "Flow in Planar Graphs with Vertex Capacities" by Khuller and Naor in Volume 11 of the journal *Algorithmica*. Finally (as with any list of records of this length), it is possible that a few of the lines in the file are misformatted.

You should start by only keeping lines for which the `year` is in the interval $[1985, 2005]$ (including the two endpoints 1985 and 2005). From this set of lines, you should construct a coauthorship network as follows.

- There should be one node for each person. (Note that even if a person is an author on 50 of the papers listed in the bibliography file, there should still just be one node corresponding to him or her, not 50.)
- There should be an undirected edge between nodes A and B if and only if they are coauthors on a paper in the bibliography. (If they are coauthors on multiple papers, there should still just be a single edge joining them.)

For example, if the file consisted of just the three lines

```
1992 27 BAMS Alon & Kleitman, Piercing Convex Sets
1994 11 ALGRTHMICA Khuller & Naor, Flow in Planar Graphs with Vertex Capacities
1996 45 IEEETC Azar & Naor & Rom, Routing Strategies for Fast Networks
```

then the graph should have node set

$$\{Alon, Azar, Khuller, Kleitman, Naor, Rom\}$$

and edge set

$$\{(Alon, Kleitman), (Azar, Naor), (Azar, Rom), (Naor, Rom), (Khuller, Naor)\}.$$

Caveats. Before we move on to the problems themselves, here are three points worth mentioning about the network we're studying here.

(i) As we'll see at various points in the course, coauthorship networks are a popular kind of "model system" for large-scale network analysis. This is not so much because there's universal fascination with the coauthoring habits of scientists (though it's an interesting topic that some people study as their research area), but because coauthorship networks are a kind of social network, encoding a particular type of collaboration among people, for which extremely rich and detailed data is available. As a result, it is a chance to try out network analysis techniques at very high resolution, in a setting that possesses many of the properties exhibited by much "messier" and harder-to-measure social networks as well.

(ii) Any time one tries to build a network from a file containing a list of names, there's the concern that different people can have the same name, and hence these different people are being "merged" into a single node. This is definitely something to worry about when one tries to draw inferences about social structure from the resulting network. However, in our case, we are using this dataset simply to build an interesting graph on which to practice various analysis techniques, so for our limited purposes there's no problem: if two authors have the same last name, then for us they *are* the same person.

(ii′) In fact, because of the issue in (2), there are papers where someone appears to coauthor with themselves. We will omit from the network those edges that link some node to itself.

(iii) Point (ii) is a particular instance of a broader principle, that in building a network from everyday data such as this list of papers, some of the entries will be misformatted or contain idiosyncracies that conflict with the general assumptions we're making about how the data is structured. For these unusual cases, it's fine to make up a consistent set of rules for how you're handling situations that don't follow the expected structure, and to document the choices you made in doing this. The goal is to make choices that don't have a significant effect on the result, where possible.

What to hand in

You should upload the following files to Canvas; please read this section carefully, since the format is important. In particular, for the first file, there is a specific line format we need, since we will be using scripts as part of the grading.

The files to hand in:

(1) An ascii .txt file named "`hw1solution.txt`". This should have results for the questions below, with each line on which you are reporting part of the answer beginning with a "`@`". The form for these lines will be described in the questions below. *The first line of this file* `hw1solution.txt` *should be just your NetID, as a single string.*

(2) Four files named "`plot1`", "`plot2`", "`plot3`", and "`plot4`", containing the plots associated with Questions 1, 2, 3, and 4 respectively. These can be in any standard image format (e.g. `plot1.png`, or `plot1.jpg`, and so forth.).

(3) The source code you used to compute the answers. By default, we won't be grading the quality of the code itself, but it will be useful to have it in case we run into any confusion. It is fine to use packages or software specifically designed for handling graphs, in which case you should just include what you wrote. If you answer the questions by some means where the notion of "source code" doesn't exactly apply, such as an interactive session with a software package, then submit whatever analogue of source code we'd need to see how you answered the questions — for example, a script you wrote as part of some larger existing package, or a transcript of an interactive session in which you did it. (This file should be named "`code`" and can be in any format; if you need to bundle together multiple files, for example, it can be a zipped or gzipped folder or tar file.)

(4) A brief description of how to apply your code (or code analogue, or transcript) to the data, together with any decisions you made about how to handle the data that would be useful for us to know about. (This file should be named "`explanation`" and can be in any format. It's this file that can address any decisions on data-handling that you had to make in accordance with general caveat (iii) above.)

Again, for most of the solutions, we'll simply be evaluating (1) and (2), and only consulting (3) or (4) as background if necessary.

The Problems

(1) Recall that the *degree* of a node is the number of edges it's incident to. We start by considering how the degrees of the nodes are distributed.

Thus, for a number j, let n_j denote the number of nodes with degree exactly j. Let d^* be the maximum degree of any node in the network. (This is the maximum total number of co-authors that any one author has — the maximum j for which $n_j > 0$.)

(a) For each j from 0 to d^*, output the number n_j. Each of these should correspond to a line in the file `hw1solution.txt` with the following four fields

`@ 1` j n_j

(The second field here simply denotes that you're answering the first question.) So for example, in the file above consisting only of the three lines

```
1992 27 BAMS Alon & Kleitman, Piercing Convex Sets
1994 11 ALGRTHMICA Khuller & Naor, Flow in Planar Graphs with Vertex Capacities
1996 45 IEEETC Azar & Naor & Rom, Routing Strategies for Fast Networks
```

the correct output would be

```
@ 1 0 0
@ 1 1 3
@ 1 2 2
@ 1 3 1
```

(Recall that before any of these lines, the first line of your file `hw1solution.txt` *should be your NetID.)*

(b) Produce a scatterplot in the plane of the ordered pairs $(\log j, \log n_j)$ for those j such that both $j > 0$ and $n_j > 0$. Hand this in as the file `plot1`. Later in the course, we'll see some proposed explanations for why such scatterplots can often be approximated fairly well by a straight line.

(2) Now we consider the sizes of the connected components in the network.

(a) Let n^* be the number of nodes in the largest connected component, and let n be the number of nodes in graph overall. Report these two quantities and their ratio, as a line in the file `hw1solution.txt` of the form

`@ 2` n^* n n^*/n

For example, on our sample graph above, you would report

```
@ 2 4 6 .667
```

Looking at the ratio of these two quantities is a good way to assess whether we should think of the network as having a "giant" component, or whether it consists entirely of small components.

(b) Let k_j denote the number of connected components of size j, and let c^* denote the size of the second-largest component. For each j from 1 to c^*, output the number k_j. Each of these should correspond to a line in the file `hw1solution.txt` with the following four fields

`@ 2` j k_j

For example, on our sample graph above, you would report

```
@ 2 1 0
@ 2 2 1
```

(c) Produce a scatterplot in the plane of the ordered pairs $(\log j, \log k_j)$ for those j such that both $1 \leq j \leq c^*$ and also $k_j > 0$. Hand this in as the file `plot2`. The extent to which logarithmic plots of component sizes should look like straight lines is less heavily studied, but there is evidence for this as well.

(3) We next consider node-to-node distances in the largest component.

(a) We start by fixing the author name `Gries` (i.e. David Gries, an eminent faculty member of Cornell CS, who joined in its early years) as our "root node." For each j, let r_j denote the number of nodes at distance exactly j from `Gries`. (So $r_0 = 1$, and r_1 is equal to the degree of `Gries`.) Let s^* denote the largest j for which $r_j > 0$ — this is the farthest anyone in the bibliography is from `Gries`, yet still connected to him by a path.

For each j from 1 to s^*, output the number r_j. Each of these should correspond to a line in the file `hw1solution.txt` with the following four fields

`@ 3` j r_j

For example, on our sample graph above, if the starting node were `Khuller` (rather than `Gries`), you would report

```
@ 3 1 1
@ 3 2 2
```

(b) Produce a histogram that plots r_j as a function of j, for j from 1 to s^*. Hand this in as the file `plot3`.

(4) Finally, we continue the analysis from the previous question by considering the structure of breadth-first search trees. In a graph G, a breadth-first search tree rooted at a node r is simply a tree T that we build as follows. We first make node r the root of T. Then, for $j = 1, 2, 3, ...$ in increasing order, we include in T all the nodes whose distance from r in G is equal to j. When we get to a value of j (meaning that we've already included all nodes at distance up to $j - 1$ in T), we consider each node v whose distance from r is equal to j. For each such v, we include it in T by making it the child of some node u at distance $j - 1$ from r, where u and v are neighbors in G. Note that since there might be multiple choices of u for a given v, this means that there may be multiple breadth-first search trees rooted at r.

A breadth-first search tree thus satisfies the following properties.

(i) The root of T is r, and T contains precisely the nodes in r's connected component.

(ii) Each edge of T is also in G.

(iii) For each node v in T, if the shortest-path distance from r to v in G is equal to j, then v is at depth j in T.

(a) Construct a breadth-first search tree T rooted at the node corresponding to `Gries`. Above, we noted why there can be many different breadth-first search trees with a given root: each time a node v at distance j from the root has multiple neighbors at distance $j-1$, each of these neighbors is eligible to be the parent of v.

One way to quantify this variability is to count the number of options that nodes have for their parents. For a node v at distance j from the root, let's say that u is a *potential parent* associated with v if u is at distance $j-1$ from the root, and u is a neighbor of v in the graph G. Note that the parent of v in the breadth-first search is one of the potential parents associated with v.

For each j, let p_j denote the average number of potential parents associated with nodes at distance j from the root node (in this case, `Gries`). That is, each node v at distance j from the root has a number of potential parents associated with it; we take the average of these numbers over all nodes v at distance j from the root.

Recall from the previous question that s^* denotes the largest j for which there is some node at distance j from `Gries`.

For each j from 1 to s^*, output the number p_j. Each of these should correspond to a line in the file `hw1solution.txt` with the following four fields

`@ 4` j p_j

To construct an example with respect to potential parents, suppose we add two more papers to our sample input from earlier (to go along with the three we already have), so that the full input is now:

```
1992 27 BAMS Alon & Kleitman, Piercing Convex Sets
1994 11 ALGRTHMICA Khuller & Naor, Flow in Planar Graphs with Vertex Capacities
1996 45 IEEETC Azar & Naor & Rom, Routing Strategies for Fast Networks
1987 16 SICOMP Azar & Vishkin, Tight Comparison Bounds on the Complexity of
Parallel Sorting
1992 24 STOC Khuller & Vishkin, Biconnectivity Approximations and Graph Carvings
```

Now, if we construct a breadth-first search tree with `Khuller` as the root node, using the co-authorship graph built from these five papers (the two above and the three original ones), then the resulting tree will have `Naor` and `Vishkin` at depth 1, and `Azar` and `Rom` at depth 2. `Naor`, `Vishkin`, and `Rom` each have one potential parent in any breadth-first search tree rooted at `Khuller`, but `Azar` has two potential parents: `Naor` and `Vishkin`.

Thus, on this example input with five papers, the output would be

```
@ 4 1 1
@ 4 2 1.5
```

(b) Produce a histogram that plots p_j as a function of j, for j from 1 to s^*. Hand this in as the file `plot4`.